Version corrigée du 2026-03-29

20/03/2026

DS Informatique 3

Vous disposez de 3 h. Les calculatrices et tout autre appareil électronique ou document de cours sont interdit. Une importance particulière sera accordée à la qualité de la rédaction : orthographe, clarté, concision, présentation. Les programmes python doivent être commentés et indentés.

I – Un jeu d'argent

On considère le jeu suivant.

Chacun des deux joueurs met successivement sur la table une quantité d'argent égale à un, deux, ou cinq euros. Le joueur qui gagne est celui qui ajoute de l'argent de façon que la somme totale atteigne un nombre au carré, soit 4, 9, etc. ou atteigne un nombre maximal, que l'on prend ici égal à 9. Le jeu s'arrête donc lorsque la somme d'argent atteint 4 ou 9. Un joueur n'est pas autorisé à dépasser 9 lors de sa mise.

Modélisation du jeu — Nous commençons par modéliser le jeu avec les outils des graphes.

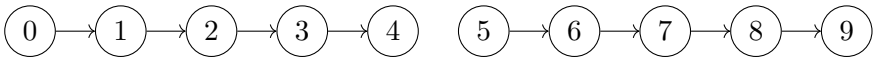
1) Quels adjectifs parmi les suivants s'appliquent à ce jeu : coopératif, non coopératif, simultané, séquentiel, à information totale, à information partielle, jeu d'accessibilité ? Justifier.

Solution

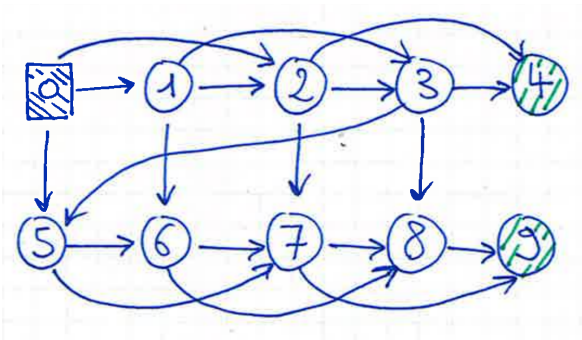
Jeu non coopératif, séquentiel, à information totale, d'accessibilité.

2) On représente ci-dessous le graphe orienté du jeu, noté G, où seuls les arcs correspondant à l'ajout d'un euro ont été représentés. Les sommets sont étiquetés par la somme d'argent sur la table. Ajouter les arcs correspondant à l'ajout de deux et cinq euros.

On tachera de rendre le graphe le plus lisible possible.



Solution



3) Définir l'origine s_0 du graphe et l'ensemble des sommets finaux F et les colorer sur le graphe précédent. Définir l'ensemble des sommets S et l'ensemble des arcs A. Ce graphe G est-il biparti ?

Solution

L'origine est $\boxed{s_0 = 0}$.

L'ensemble des sommets finaux est $\boxed{F = \{4, 9\}}$.

L'ensemble des sommets est $\boxed{S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}}$.

L'ensemble des arcs est

$$\boxed{\begin{aligned} A = \{&(0,1),(0,2),(0,5),(1,2),(1,3),(1,6),(2,3),(2,4),(2,7),(3,4),(3,5),(3,8),(5,6),(5,7),\\ &(6,7),(6,8),(7,8),(7,9),(8,9)\}. \end{aligned}}$$

Le graphe G n'est $\boxed{\text{pas biparti}}$ *car il existe des cycles (non orientés) impairs, par exemple le cycle $0 \to 1 \to 2 \leftarrow 0$, ce qui interdit d'avoir une bipartition $S = S_1 \cup S_2$ tels que les arcs relient uniquement les sommets de S_1 avec ceux de S_2.*

4) Implémenter le graphe G dans le langage `python` sous forme d'une liste d'adjacence, représentée par un dictionnaire `G`. On rappelle la syntaxe d'un dictionnaire : `{clé1:valeur1,clé2:valeur2,...}`

Solution

```
G = {0:[1,2,5],1:[2,3,6],2:[3,4,7],3:[4,5,8],4:[],5:[6,7],6:[7,8],7:[8,9],8:[9],9:[]}
```

5) Quelle propriété possède l'origine s_0 vis-a-vis du dictionnaire `G` ? Compléter les parties pontillés du code suivant pour définir la fonction `origine` qui prend en argument le dictionnaire `G` et renvoie l'origine du graphe. On pourra utiliser la méthode `liste.remove(element)` qui retire `element` de `liste`. Commenter toutes les lignes du code. *Attention : la commande `liste.remove(element)` renvoie une erreur si `element` n'est pas dans `liste` !*

```python
def origine(G):
    """
    Entrée : G (dict)
    Sortie : origine de G (int)
    """
    S = G.keys()
    for s in S:
        for ... in ...:
            ...
    return S[0]
```

Solution

L'origine est sans prédécesseurs, donc $\forall s \in S, s_0 \notin G[s]$. On en déduit la fonction `python` suivante.

```python
def origine(G):
    """
    Entrée : G (dict)
    Sortie : origine de G
    """
    S = [s for i in G]        # Liste des sommets du graphe
    for s in G:               # Parcours des sommets du graphe
        for p in G[s]:        # Si p est un successeur de s
            if p in S:        # Si p est encore dans la liste des sommets
                S.remove(p)   # On retire p de la liste des sommets
    return S[0]               # On retourne l'origine de G
```

6) Effectuer l'exécution pas-à-pas de l'appel `origine(G)` sur le dictionnaire `G` défini précédemment, en indiquant la valeur initiale de la liste `S`, puis la valeur des variables `s`, `G[s]`, et `S` à l'issue de chaque itération dans la boucle, et enfin la valeur de `S[0]` retournée par la fonction.

Solution

La valeur initiale de `S` *est* `[0,1,2,3,4,5,6,7,8,9]`.

s	G[s]	S(avant)	S(après)
0	[1,2,5]	[0,1,2,3,4,5,6,7,8,9]	[0,3,4,6,7,8,9]
1	[2,3,6]	[0,3,4,6,7,8,9]	[0,4,7,8,9]
2	[3,4,7]	[0,4,7,8,9]	[0,8,9]
3	[4,5,8]	[0,8,9]	[0,9]
4	[]	[0,9]	[0,9]
5	[6,7]	[0,9]	[0,9]
6	[7,8]	[0,9]	[0,9]
7	[8,9]	[0,9]	[0]
8	[9]	[0]	[0]
9	[]	[0]	[0]

La fonction renvoie alors

$$\boxed{S[0] = 0}$$

7) Quelle propriété possède les sommets finaux vis-a-vis du dictionnaire `G` ? Implémenter une fonction `final` qui prend en argument le dictionnaire `G` et renvoie la liste `F` des sommets finaux. Les spécifications de la fonction seront donnés et le code sera commenté.

Solution

Les sommets finaux sont sans successeurs, soit $G[s] = [\,], \forall s \in F$. On en déduit la fonction `python` suivante.

```python
def final(G):
"""
Entrée : G, un dictionnaire représentant un graphe orienté
Sortie : F, une liste des sommets finaux
"""
F = []              # Initialisation de la liste des sommets finaux
for s in G:         # Parcours des sommets du graphe
    if G[s] == []:  # Si le sommet s n'a pas de successeur
        F.append(s)   # On ajoute s à la liste des sommets finaux
return F            # On retourne la liste des sommets finaux
```

Proposition de stratégies — Dans la suite, on ne représentera pas le graphe biparti associé à G mais on gardera à l'esprit qu'un même sommet $0 \leqslant i \leqslant 9$ peut représenter deux configurations différentes du jeu suivant que le sommet appartient au premier joueur J_1 ou au second joueur J_2.

8) Déterminer quels sommets peuvent appartenir à J_1 ou J_2 et définir les ensembles correspondant S_1 et S_2. Attention, certains sommets n'appartiennent qu'à un seul des joueurs, tandis que d'autres peuvent appartenir aux deux joueurs.

Solution

$\boxed{S_1 = \{0,2,3,4,5,6,7,8,9\}}$ *et* $\boxed{S_2 = \{1,2,3,4,5,6,7,8,9\}}$. *Le sommet* 0 *n'appartient qu'à* J_1 *tandis que* 1 *n'appartient qu'à* J_2.

9) On définit la fonction $f : S_1 \setminus F \to S_2$ telle que $f(0) = 5$, $f(2) = 4$, $f(3) = 4$, $f(5) = 7$, $f(6) = 8$, $f(7) = 9$, et $f(8) = 9$. Cette fonction définit-elle une stratégie ? Si oui, cette stratégie est-elle gagnante pour le joueur J_1 ? Justifier.

Solution

L'ensemble de départ est bien défini sur S_1/F *car* $1 \notin S_1$ *et* $F = \{4,9\}$. *L'ensemble d'arrivée est bien défini car* $\{1,5,4,8,9\} \cap S_2$.

La fonction f *définit bien une* $\boxed{\text{stratégie}}$ *car les paires* $(0,1)$, $(2,4)$, $(3,4)$, $(5,7)$, $(6,8)$, $(7,9)$, *et* $(8,9)$ *sont bien des arcs.*

La stratégie f *n'est* $\boxed{\text{pas gagnante}}$ *pour le joueur* J_1. *En effet, la partie* $0 \xrightarrow{J_1} 5 \xrightarrow{J_2} 6 \xrightarrow{J_1} 8 \xrightarrow{J_2} 9$ *est jouée suivant* f *mais perdante pour* J_1 *car le joueur* J_2 *passe de* 6 *au sommet final* 9.

10) On définit la stratégie $g : S_2 \setminus F \mapsto S_1$ telle que $g(1) = 6$, $g(2) = 4$, $g(3) = 4$, $g(5) = 6$, $g(6) = 7$, $g(7) = 9$, et $g(8) = 9$. Montrer que cette stratégie est gagnante pour le second joueur J_2.

Solution

La stratégie g *est* $\boxed{\text{gagnante}}$ *pour le joueur* J_2. *En effet, toutes les parties jouées suivant* g *sont de la forme* $0 \xrightarrow{J_1} 1,2,5 \xrightarrow{J_2} 4,6(\xrightarrow{J_1} 7,8 \xrightarrow{J_2} 9)$ *et sont gagnantes pour* J_2.

Algorithme min-max — Pour déterminer de façon certaine s'il existe ou non une stratégie gagnante pour l'un ou l'autre des joueurs, nous appliquons l'algorithme min-max.

11) Rappeler le principe de l'algorithme min-max. Rappeler l'avantage de la représentation en arbre par rapport à la représentation en graphe.

Solution

> *Voir cours. Un graphe représente toutes les configurations du jeu possibles ; une arbre représente toutes les parties possibles.*

12) Représenter l'arbre de décision associé à G en indiquant à quel joueur appartiennent les sommets de chaque étage de l'arbre, puis appliquer l'algorithme min-max à la main sur l'arbre de décision. En déduire s'il existe une stratégie gagnante pour J_1 ou pour J_2.

13) On implémente la fonction `minmax` suivante. Compléter les parties pointillées du code en utilisant les méthodes `max` et `min` de python, et commenter chaque ligne du code.

```python
def minmax(G,s):
    """
    Entrée : graphe G (dictionnaire), sommet initial s
    Sortie : heuristique h(s) (entier)
    """
    if G[s] == []:
        if s[0] == 1:
            return ...
        else:
            return ...
    else:
        h = [minmax(G,v) for v in G[s]]
        if s[0] == 1:
            return ...
        else:
            return ...
```

Solution

```python
def minmax(G,s):
    """
    Entrée : graphe G (dictionnaire), sommet initial s = (j,i)
    Sortie : heuristique h(s) (entier)
    """
    if G[s] == []:
        if s[0] == 1:
            return -1
        else:
            return +1
    else:
        h = [minmax(G,v) for v in G[s]]
        if s[0] == 1:
            return max(h)
        else:
            return min(h)
```

Algorithme du noyau — On souhaite vérifier la conclusion précédente par un l'algorithme du noyau. Le *noyau* d'un graphe orienté G est le sous-ensemble de sommets $N \subset S$ tel que :

(i) aucun arc ne joint deux sommets de N : $\forall s_1, s_2 \in N, (s_1, s_2) \notin A$,

(ii) tout sommet hors de N a au moins un successeur dans N : $\forall s_1 \in S \setminus N, \exists s_2 \in N, (s_1, s_2) \in A$.

14) On peut construire le noyau récursivement. Soit G un graphe dont l'ensemble des sommets finaux est F. On initialise le noyau par l'ensemble vide puis on procède de la manière suivante :

ⓐ on choisit un sommet final $s \in F$, que l'on ajoute au noyau N ;

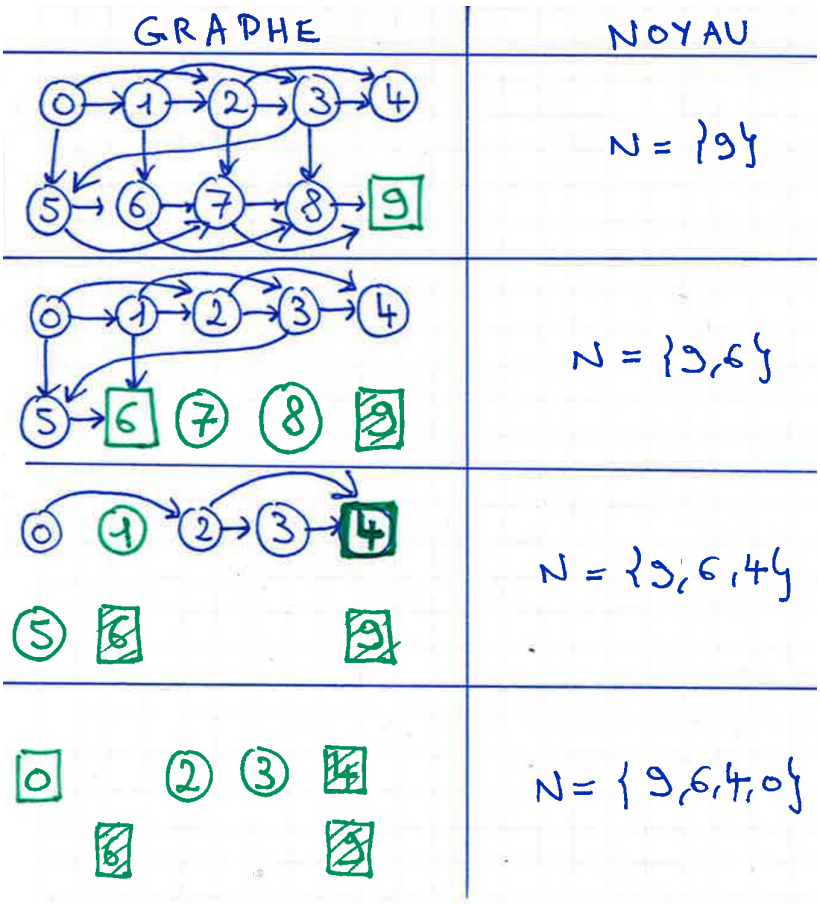
ⓑ on retire de G tous les prédécesseurs p de s, ainsi que tous les arcs dont p ou s est une extrémité ;

ⓒ on répète le processus sur le sous-graphe obtenu jusqu'à ce que le sous-graphe soit vide.

Appliquer cet algorithme sur le graphe G précédent à partir du sommet final $s = 9$ en représentant à chaque étape le sous-graphe obtenu et le contenu du noyau. Montrer que le noyau du graphe G est l'ensemble $N = \{0, 4, 6, 9\}$.

Solution

Lors de la deuxième étape, le sous-graphe contient deux sommets finaux : 4 et 6. On peut choisir l'un et l'autre de ces sommets pour continuer la procédure.



15) On se propose d'implémenter l'algorithme du noyau en deux étapes. Pour commencer, on souhaite définir une fonction `retrait` qui retire une liste de sommets `L` d'un graphe `G`, en mettant à jour les listes d'adjacence. Compléter les parties pointillées du code suivant en utilisant la méthode `remove` de python et commenter chaque nouvelle ligne du code.

```
def retrait(G,L):
    for s in L:      # Pour tout sommet s à retirer
        ...          # Retirer s de la liste des sommets
        for p in G: # Pour tout sommet p du graphe
            ...
```

Solution

```
def retrait(G,L):
    for s in L:                 # Pour tout sommet s à retirer
        G.remove(s)             # Retirer s de la liste des sommets
        for p in G:             # Pour tout sommet p du graphe
            if s in G[p]:       # Si s successeur de p
                G[p].remove(s)  # Retirer s de la liste des successeurs de p
```

Veiller à bien mettre la condition `if s in G[p]` pour éviter que le code ne renvoie une erreur si s n'est pas dans la liste des successeurs de p. La méthode `remove` s'applique à un dictionnaire comme à un graphe. Le graphe G est-il modifié par cette fonction ?

16) On propose maintenant de définir une fonction récursive `noyau` à partir du pseudo-code suivant.

Si G dictionnaire vide
  | **Retourner** N
$f \leftarrow$ sommet final
$L \leftarrow$ liste $[f]$
Pour s dans G
  | **Si** f successeur de s
  | | **Ajouter** s à L
retrait(G, L)
noyau$(G, N + [f])$

Implémenter la fonction `noyau(G,N)` en python en précisant les spécifications et en commentant le code. On pourra se servir de la fonction `final` définie précédemment. Effectuer l'exécution pas à pas de l'appel `noyau(G,[])` et comparer aux résultats précédents.

Solution

17) On admet que le noyau N contient l'ensemble des *positions perdantes pour* J_1. Le complémentaire du noyau dans S donne au contraire les *positions gagnantes pour* J_1. Conclure sur l'existence d'une stratégie gagnante pour J_1 ou J_2.

Solution

Il existe une stratégie gagnante pour J_2 mais pas pour J_1.

18) Quel autre objet de la théorie des graphes similaire au noyau connaissez-vous qui permette également de trouver les positions gagnantes ? Rappeler sa définition générale sans chercher à déterminer cet objet dans cet exemple précis.

Solution

L'attracteur, voir cours.

II – Classement de fruits

On dispose d'un ensemble de données sur divers fruits caractérisés par leur masse et leur couleur. La masse est indiquée en grammes et représentée par un entier positif, tandis que la couleur est représentée par un flottant compris entre 0 et 1 qui indique si la couleur est plutôt « chaude » ou « froide », suivant la correspondance suivante :

Couleur	Valeur
Rouge	0.85 - 1.00
Orange	0.75 - 0.85
Jaune	0.65 - 0.75
Vert	0.45 - 0.65

On souhaite classer ces de nouvelles données de fruits à l'aide de l'algorithme des k-moyennes, connaissant uniquement les données de masse et de couleur. On dispose pour cela d'un ensemble d'apprentissage, représentée par une liste `parametres` dont les premières entrées sont `parametres = [(123,0.81), (150,0.73), (100,0.65), ...]`, et d'une liste `fruits` de même longueur contenant les étiquettes de chaque fruit associées à `parametres`, et qui commence par `fruits = ['tomate', 'banane', 'kiwi', ...]`. L'ensemble de ces données est représenté sur le graphique de la Fig. II.1 ci-dessous.

19) Le programmeur ne connaît pas encore l'ensemble des fruits représentés dans les données, et donc le nombre de classes à former. Reproduire, compléter et commenter la fonction `classes_fruits` ci-dessous qui prend comme argument la liste `fruits` et qui renvoie l'ensemble des classes, i.e la liste de tous les fruits possibles.

Les pointillés ... peuvent représenter plusieurs lignes de code.

```
def classes_fruits(fruits):
    """
    Entrée : fruits (list) : liste de chaînes de caractères
```

```
    Sortie : classes (list) : liste de chaînes de caractères
    """
    classes = []
    ...
    return classes
```

Solution

```
def classes_fruits(fruits):
    """
    Entrée : fruits (list) : liste de chaînes de caractères
    Sortie : classes (list) : liste de chaînes de caractères
    """
    classes = []
    for fruit in fruits:
        if fruit not in classes:
            classes.append(fruit)
    return classes
```

20) Reproduire, compléter et commenter la fonction `classification` ci-dessous qui prend comme arguments les listes `fruits` et `parametres`, et qui renvoie un dictionnaire contenant le classification de l'ensemble d'apprentissage, i.e. tel que chaque entrée est de la forme `(masse, couleur) : fruit`.
Les pointillés ... peuvent représenter plusieurs lignes de code.

```
def classification(fruits, parametres):
    """
    Entrée : fruits (list) : liste de chaînes de caractères
             parametres (list) : liste de tuples (masse, couleur)
    Sortie : p (dict) : dictionnaire contenant la classification
    """
    p = {}
    ...
    return p
```

Solution

```
def classification(fruits, parametres):
    """
    Entrée : fruits (list) : liste de chaînes de caractères
             parametres (list) : liste de tuples  (masse, couleur)
    Sortie : p (dict) : classification, de type (masse, couleur) : fruit
    """
    p = {}
    for i in range(len(fruits)):
        p[parametres[i]] = fruits[i]
    return p
```

Pour la distance, nous prenons une distance euclidienne dans l'espace (masse, couleur). Comme ces deux grandeurs sont de dimensions différentes, on peut pondérer ces deux grandeurs par des coefficients différents, de façon à donner un poids équitable à chaque paramètre. La couleur est déjà normalisée entre 0 et 1, tandis que la masse peut varier entre 0 et 300 grammes typiquement. Nous normaliserons donc la masse par l'unité de masse maximale fixée conventionnellement à 300 grammes.

21) Reproduire, compléter et commenter la fonction `distance` ci-dessous qui prend comme arguments deux points du plan de coordonnées (m_1, c_1) et (m_2, c_2), et qui renvoie la distance euclidienne pondérée entre ces deux points.

```
def distance(p1, p2):
    """
    Entrée : p1 (tuple) : un point du plan de coordonnées (masse, couleur)
             p2 (tuple) : un point du plan de coordonnées (masse, couleur)
    Sortie : distance euclidienne pondérée entre p1 et p2
    """
```

```
    m1, c1 = p1
    m2, c2 = p2
    return ...
```

Solution

```
def distance(p1, p2):
    """
    Entrée : p1 (tuple) : point du plan de coordonnées (masse, couleur)
             p2 (tuple) : point du plan de coordonnées (masse, couleur)
    Sortie : d (float) : distance euclidienne pondérée entre p1 et p2
    """
    m1, c1 = p1
    m2, c2 = p2
    d = ((m1/300 - m2/300)**2 + (c1 - c2)**2)**0.5
    return d
```

Nous appliquons maintenant l'algorithme des k plus proches voisins.

22) Reproduire, compléter et commenter la fonction `kvoisins` ci-dessous, qui prend comme arguments un point du plan de coordonnées (m, c), l'ensemble d'apprentissage `parametres`, la classification `p`, la distance `distance`, et un entier k, et qui renvoie la classe majoritaire parmi les k plus proches voisins de ce point. On précisera les spécifications de cette fonction et on la commentera.
Les pointillés ... peuvent représenter plusieurs lignes de code.

```
def kvoisins(point, parametres, p, distance, k):
    """
    Entrée : point (tuple) : point du plan (masse, couleur)
        parametres (list) : liste de tuples
        p (dict) : dictionnaire contenant la classification
        distance (func) : fonction distance entre deux points du plan
        k (int) : nombre de plus proches voisins
    Sortie : classe (string) : classe majoritaire parmi les k plus proches voisins
    """
    tri = sorted(parametres, key=lambda p: distance(point, p))
    voisins = tri[:k]
    classes_count = {}
    for voisin in voisins:
        ...
    classe_majoritaire = max(classes_count, key=lambda c: classes_count[c])
    return classe_majoritaire
```

Solution

```
def kvoisins(point, parametres, p, distance, k):
    tri = sorted(parametres, key=lambda p: distance(point, p))
    # On prend les k plus proches voisins
    voisins = tri[:k]
    # On compte le nombre de voisins dans chaque classe
    classes_count = {}
    for voisin in voisins:
        classe_voisin = p[voisin]
        if classe_voisin in classes_count:
            classes_count[classe_voisin] += 1
        else:
            classes_count[classe_voisin] = 1
    # On trouve la classe majoritaire
    classe_majoritaire = max(classes_count, key=classes_count.get)
    return classe_majoritaire
```

23) On représente l'ensemble d'apprentissage `parametres` sur la Fig. II.1 ci-dessous, pour un exemple de trois fruits : des tomates, des bananes, et des kiwis. On applique la fonction `kvoisins` à un fruit de coordonnées normalisées $x = (0.77, 0.78)$. En s'aidant d'un compas ou d'une règle, déterminer graphiquement la classe majoritaire de cette nouvelle donnée pour $k = 1, 3$ et 7 plus proches voisins, en expliquant votre construction. Commenter les résultats obtenus.

Pensez à rendre l'énoncé avec le graphique complété avec votre copie !

Solution

Pour $k = 1$, le voisin le plus proche de x est un point de classe « Tomate », donc la classe majoritaire est « Tomate ».
Pour $k = 3$, les trois voisins les plus proches de x sont deux points de classe « Banane » et un point de classe « Tomate », donc la classe majoritaire est « Banane ».
Pour $k = 7$, les sept voisins les plus proches de x sont quatre points de classe « Tomate » et trois points de classe « Banane », donc la classe majoritaire est « Tomate ».
On remarque que le résultat de l'algorithme des k plus proches voisins dépend du choix de k !

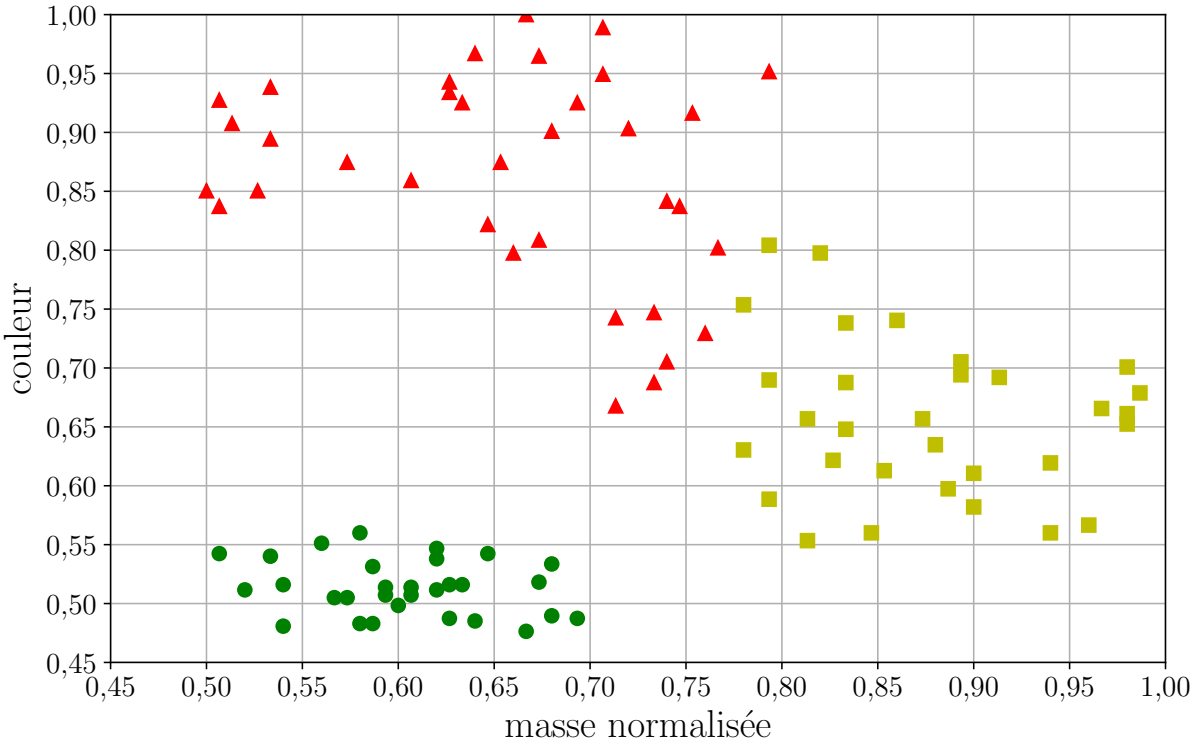


Fig. II.1 – Ensemble d'apprentissage de fruits représenté dans le plan (masse, couleur).

On souhaite améliorer la prédiction de l'algorithme des k plus proches voisins en pondérant la classe majoritaire par le nombre de voisins appartenant à chaque classe.le classes des voisins suivant la distance au nouveau point x. Notons $V_i(x, k)$ le sous-ensemble des voisins de x de classe i parmi les k plus proches voisins de x. On propose d'attribuer à chaque classe un poids égal à

$$\sigma_i = \sum_{v \in V_i(x,k)} d(x, v)^{-2}. \tag{II.1}$$

24) On donne la liste triée des 7 premiers voisins de la donnée $x = (0.77, 0.78)$ ci-dessous, avec leur classe et leurs coordonnées normalisées. Calculer les poids σ_i de chaque classe « Tomate » ou « Banane » pour les $k = 1, 3$ et 7 plus proches voisins de x.

Solution

Voisin	*1*	*2*	*3*	*4*	*5*	*6*	*7*
Classe	*Tomate*	*Banane*	*Banane*	*Tomate*	*Tomate*	*Tomate*	*Banane*
Coordonnées	$(0.76, 0.80)$	$(0.79, 0.80)$	$(0.78, 0.75)$	$(0.75, 0.84)$	$(0.76, 0.73)$	$(0.74, 0.84)$	$(0.82, 0.80)$
$d(v, x)^{-2}$	2000	1250	1000	250	385	222	345

On calcule le poids des classes pour chaque valeur de k.

k	σ_{Tomate}	σ_{Banane}
1	2000	0
3	2000	2250
7	2857	2595

Voisin	1	2	3	4	5	6	7
Classe	Tomate	Banane	Banane	Tomate	Tomate	Tomate	Banane
Coordonnées	$(0.76, 0.80)$	$(0.79, 0.80)$	$(0.78, 0.75)$	$(0.75, 0.84)$	$(0.76, 0.73)$	$(0.74, 0.84)$	$(0.82, 0.80)$

25) On choisit d'attribuer à la nouvelle donnée x la classe dont le poids est le plus élevé. Quelle classe est attribuée à x pour $k = 1, 3$ et 7 voisins ? Commenter les résultats obtenus.

Solution

On trouve comme résultats :

k	*Classe attribuée à x*
1	*Tomate*
3	*Banane*
7	*Tomate*

On voit que pour $k = 3$, les tomates et les bananes ont des poids assez proches, mais toujours au profit des bananes. Malgré l'amélioration de la prédiction grâce à la pondération, le résultat de l'algorithme des k plus proches voisins dépend toujours du choix de k. La nouvelle donnée x est située à la frontière entre les classes « Tomate » et « Banane », ce qui rend sa classification plus délicate.